

Unit 1: 스레드 모델 기초

강사: 조영민

목표

- 스레드의 개념과 프로세스와의 차이를 이해한다
- Thread, ThreadPool, Task.Run 세 가지 실행 모델의 차이와 적절한 사용 시점을 판단한다
- Foreground/Background 스레드가 프로세스 수명에 미치는 영향을 설명할 수 있다
- [ThreadStatic] 과 TLS(Thread Local Storage)의 동작 원리와 초기화 함정을 파악한다

프로세스 vs 스레드 (1/2)

구분	프로세스 (Process)	스레드 (Thread)
메모리	독립된 가상 주소 공간	같은 프로세스 내 힙/데이터 공유
생성 비용	높음 (OS 자원 할당)	낮음 (스택만 할당)
통신	IPC 필요 (파이프, 소켓 등)	메모리 직접 공유

프로세스 vs 스레드 (2/2)

구분	프로세스 (Process)	스레드 (Thread)
장애 격리	프로세스 간 격리됨	하나의 스레드 예외가 프로세스 전체에 영향
단위	OS 자원 할당의 단위	CPU 스케줄링의 단위

하나의 프로세스는 최소 1개의 스레드(Main 스레드)를 가진다.

Thread 생성과 Join

Thread 객체를 생성하고 Start() 로 시작, Join() 으로 완료를 대기한다.

```
Thread t1 = new Thread(ComputeSum) { Name = "Worker-1" };
Thread t2 = new Thread(ComputeSum) { Name = "Worker-2" };
t1.Start(5_000_000);
t2.Start(5_000_000);
t1.Join();    // t1 종료까지 Main 블로킹
t2.Join();    // t2 종료까지 Main 블로킹
```

Thread 생성과 Join -- 핵심 포인트

- Join() : 대상 스레드가 끝날 때까지 호출 스레드를 대기(블로킹)시킨다
- 실행할 때마다 출력 순서가 달라질 수 있다 -- 스케줄링의 **비결정성**

Thread.Start(object) -- ParameterizedThreadStart (1/2)

```
static void ComputeSum(object? limitObj)
{
    int limit = limitObj is int v ? v : 1_000_000;
    long sum = 0;
    for (int i = 1; i <= limit; i++)
        sum += i;
}
```

Thread.Start(object) -- ParameterizedThreadStart (2/2)

- `Start(object)` 는 `ParameterizedThreadStart` 델리게이트를 사용한다
- `int` 등 값 타입을 `object` 로 전달하면 **boxing**이 발생한다
- `Thread.CurrentThread` : 현재 실행 중인 스레드 객체를 반환하는 정적 프로퍼티

Foreground vs Background 스레드

구분	Foreground (기본값)	Background
설정	<code>IsBackground = false</code>	<code>IsBackground = true</code>
프로세스 종료	이 스레드가 끝날 때까지 프로세스 유지	모든 포그라운드 스레드 종료 시 함께 즉시 종료
finally 실행	정상 종료 시 보장 (FailFast/StackOverflow 제외)	강제 종료 시 실행 안 될 수 있음
용도	반드시 완료해야 하는 작업	로깅, 모니터링 등 부수 작업

Foreground vs Background -- 코드 예시

```
Thread worker = new Thread(() =>
{
    Thread.Sleep(3000);
    Console.WriteLine("[Worker] 작업 완료");
})
{ IsBackground = true };
```

- `IsBackground = true` 이면 모든 포그라운드 스레드 종료 시 함께 종료된다
- `false` 로 바꾸면 3초 후 출력을 확인할 수 있다

ThreadPool 구조 -- 왜 직접 Thread를 만들지 않는가

ThreadPool은 미리 생성된 스레드를 재사용하는 풀(Pool) 구조이다.

직접 Thread 생성	ThreadPool 사용
매번 OS 스레드 생성/소멸 비용 발생	스레드를 재사용하여 비용 절감
스레드 수 제어가 어려움	런타임이 자동으로 스레드 수 조절
짧은 작업에 비효율적	짧은 작업에 최적화

ThreadPool -- 코드 예시

```
// ThreadPool에 작업을 큐잉
ThreadPool.QueueUserWorkItem(_ =>
{
    Workload.Compute();
    ev.Set();
});
```

QueueUserWorkItem : 작업을 풀 큐에 넣으면 가용 스레드가 자동으로 실행한다.

Thread vs ThreadPool vs Task.Run 비교 (1/2)

항목	Thread	ThreadPool	Task.Run
추상화 수준	최하위 (OS 직접)	중간 (풀 관리)	최상위 (async 연계)
생성 비용	높음	낮음 (재사용)	낮음 (풀 기반)
스레드 수 제어	수동	런타임 자동	런타임 자동
반환값	없음	없음	<code>Task<T></code> 반환 가능

Thread vs ThreadPool vs Task.Run 비교 (2/2)

- 일반적으로 Task.Run을 기본으로 사용하고, 특별한 이유가 있을 때만 Thread를 직접 생성한다

```
// 세 가지 방식 비교 (Lab03)
Measure("Thread",    () => { /* ... */ });
Measure("ThreadPool", () => { /* ... */ });
Measure("Task.Run",  () => Task.Run(
    Workload.Compute).Wait());
```

[ThreadStatic]과 TLS -- 스레드별 독립 저장소

[ThreadStatic] 어트리뷰트는 `static` 필드를 스레드마다 독립된 사본으로 만든다.

```
[ThreadStatic]
private static int _tlsCounter;

// 스레드 A가 ++해도 스레드 B에는 영향 없음
for (int i = 0; i < Iterations; i++)
    _tlsCounter++; // 동기화 불필요
```

[ThreadStatic] vs lock vs Interlocked

방식	동기화	성능	용도
[ThreadStatic]	불필요	가장 빠름	스레드별 독립 카운터/캐시
lock + static	Monitor 기반	느림	공유 자원 보호
Interlocked	원자적 CPU 명령	중간	공유 카운터 증감

[ThreadStatic] 초기화 함정

[ThreadStatic] 필드에 초기값을 지정해도 정적 생성자를 실행하는 스레드(보통 해당 타입에 처음 접근하는 스레드)에서만 초기화된다.

```
[ThreadStatic] static int _value = 42;  
// Main 스레드: _value == 42  
// 다른 스레드: _value == 0 (default(int))
```

ThreadLocal<T> 대안

ThreadLocal<T> 는 팩토리 함수로 모든 스레드에서 초기값을 보장한다.

```
static ThreadLocal<int> _tls = new(() => 42);  
// 모든 스레드에서 _tls.Value == 42
```

면접 TIP: "[ThreadStatic] 필드의 초기화 시점은 언제인가요?"

-- 정적 생성자에서 1회만 실행되므로, 해당 스레드(보통 Main)에서만 초기값이 반영됩니다. 다른 스레드는 default(T)입니다.

컨텍스트 스위칭과 Parallel.For

Parallel.For는 반복을 ThreadPool 스레드들에 자동 분배하여 병렬 실행한다.

```
Parallel.For(0, 20, i =>
{
    int tid = Thread.CurrentThread.ManagedThreadId;
    Console.WriteLine($"[Task {i}] Thread {tid}");
});
```

- **MaxDegreeOfParallelism** : 동시 실행 스레드 수를 제한할 수 있다

Parallel.For -- 관찰 포인트

- `ManagedThreadId` : .NET 런타임이 부여하는 스레드 고유 ID (OS 스레드 ID와 다름)
- 같은 `ManagedThreadId`가 여러 Task에 재사용되는 것을 관찰할 수 있다
- `Process.GetCurrentProcess().Threads.Count` 로 OS 수준 전체 스레드 수 확인 가능

고찰점 -- 게임 서버에서 스레드를 어떻게 쓸 것인가

- 게임 루프(Tick)는 단일 스레드로 처리하고, I/O는 ThreadPool에 위임하는 패턴이 일반적
- 스레드 수를 CPU 코어 수 이상으로 늘리면 **컨텍스트 스위칭** 비용이 증가한다
- 공유 상태를 최소화하고, 필요 시 lock 대신 **Interlocked** 또는 **lock-free 자료구조**를 검토한다

면접 TIP: Thread vs Task.Run

"Thread와 Task.Run의 차이를 설명하세요."

-- Thread는 OS 스레드를 직접 생성하므로 비용이 크고, Task.Run은 ThreadPool 위에서 동작하여 스레드 재사용과 스케줄링 최적화를 제공합니다. 반환값이 필요하거나 async/await 체인이 필요하면 Task.Run을 사용합니다.

실습 안내 (1/2)

Lab	주제	핵심 API
Lab01	Hello, Thread!	<code>Thread</code> , <code>Join()</code> , <code>Start(object)</code>
Lab02	Foreground vs Background	<code>IsBackground</code> , <code>Thread.Sleep</code>
Lab03	Thread vs ThreadPool vs Task	<code>QueueUserWorkItem</code> , <code>Task.Run</code>

실습 안내 (2/2)

Lab	주제	핵심 API
Lab04	[ThreadStatic] 벤치마크	[ThreadStatic] , Interlocked , lock
Lab05	동시성 시각화	Parallel.For , ManagedThreadId

- 모든 벤치마크는 반드시 **Release** 빌드로 실행: `dotnet run -c Release`
- Lab03/04는 `-- --benchmark` 옵션으로 BenchmarkDotNet 결과를 확인할 수 있다

핵심 정리 (1/2)

- **Thread**는 OS 스레드를 직접 생성하며 비용이 크므로, 대부분의 경우 **Task.Run** (ThreadPool 기반)을 사용한다
- **Foreground** 스레드는 프로세스 수명을 연장하고, **Background** 스레드는 모든 포그라운드 스레드 종료 시 함께 종료된다

핵심 정리 (2/2)

- [ThreadStatic] 은 스레드별 독립 저장소(TLS)를 제공하지만, 초기값은 정적 생성자를 실행하는 스레드(보통 해당 타입에 처음 접근하는 스레드)에서만 설정된다 --
ThreadLocal<T> 로 해결
- Interlocked는 lock 없이 원자적 연산을 수행하며, 단순 카운터에는 lock보다 효율적이다
- 스레드 수를 코어 수 이상으로 늘리면 **컨텍스트 스위칭** 오버헤드가 증가하므로 적절한 병렬도를 설정해야 한다